

# UniVerse

A Campus Companion

## Resources Hub

Semester folders of study materials - admin uploads files or Drive links, students browse and open them.

---

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Robi (student) + Fahmid (admin upload)

Key files: features/resources/\*, features/admin/.../manage\_resources\_screen.dart

## 1. What it does

The Resources Hub is the department's shared file drawer. **Admins upload** any file (PDF/DOC/PPT/image/zip) or paste a Google Drive link, tagged to a semester and category. **Students browse** by semester folder -> category -> and tap to open. Files live in Supabase Storage; links open in the browser.

| file                                        | job                                                                                                                                      |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>resource_service.dart</code>          | The only Supabase layer: fetch by semester, upload to the `resources` bucket, insert/delete rows.                                        |
| <code>resource_controller.dart</code>       | Student-side state: loads resources, filters by semester + category.<br>Admin-side: pick a file, upload, create the row, alert students. |
| <code>resource_admin_controller.dart</code> |                                                                                                                                          |

## 2. Why resources filter by SEMESTER (not batch+section)

This is a deliberate contrast with routines. A 'Data Structures' note is useful to every student in that semester regardless of section, so resources key on **semester**. Routines key on batch+section because a schedule is cohort-specific. Good detail to mention - it shows you chose the data model per use-case.

## 3. Upload flow (admin)

- 1 Admin picks a file (file\_picker, with bytes) or enters a Drive URL + sets semester + category.
- 2 `uploadFile(bytes, name, semester)` sanitises the filename, builds a path `<semester>/<timestamp>_<name>`, uploads to the public `resources` bucket, returns the public URL.
- 3 `createResource()` inserts the row, stamping `uploaded_by` from the session (required by the RLS insert policy).
- 4 A best-effort broadcast alerts students that new material was added.

Content-type is inferred from the extension so the browser opens the file correctly instead of downloading a blob.

## 4. Browse flow (student)

- `ResourceController` loads resources; the screen shows **semester folders** (all semesters - RLS lets students read every semester).
- Tapping a folder lists items; a category chip row filters client-side for instant response (no re-query).
- Tapping an item opens the URL via `url_launcher`, with a clipboard fallback if no browser can handle it.

## 5. Likely defense questions

| question                    | answer                                                                                      |
|-----------------------------|---------------------------------------------------------------------------------------------|
| Where are the files stored? | Supabase Storage, the public `resources` bucket; the DB row stores the URL + metadata.      |
| Why public bucket?          | MVP simplicity; access is read-only browse. Could be tightened with signed URLs later.      |
| How are uploads             | RLS insert policy requires <code>uploaded_by</code> = the caller and an admin/teacher role. |
| Do students get notified?   | Yes - upload fires a best-effort notification to students (in-app + push).                  |